



Session: 01a0263d-61ad-7b7b-a874-1fc9f7bfedb2

T toggle thinking · 0 toggle tools

Toggle thinking

Toggle tools

↓ JSONL

Date: 8/21/2026, 5:32:32 PM
Models: ollama/qwen3.6:35b-mlx, ollama/qwen3.8:27b-mlx
Messages: 2 user, 9 assistant, 10 tool results
Tool Calls: 10
Tokens: ↑90k ↓5.0k
Cost: 0.000

System Prompt

You are an expert coding assistant operating inside pi, a coding agent harness. You help users by reading files, executing commands, editing code, and writing new files.

Available tools:

- read: Read file contents
- bash: Execute bash commands (ls, grep, find, etc.)
- edit: Make precise file edits with exact text replacement, including multiple disjoint edits in one call
- write: Create or overwrite files

In addition to the tools above, you may have access to other custom tools depending on the project.

... (368 more lines, click to expand)

Available Tools

read - Read the contents of a file. Supports text files and images (jpg, png, gif, webp, bmp). Images are sent as attachments. For text files, output is truncated to 2000 lines or 50KB (whichever is hit first). Use offset/limit for large files. When you need the full file, continue with offset until complete. *[click to show parameters]*

bash - Execute a bash command in the current working directory. Returns stdout and stderr. Output is truncated to last 2000 lines or 50KB (whichever is hit first). If truncated, full output is saved to a temp file. Optionally provide a timeout in seconds. *[click to show parameters]*

edit - Edit a single file using exact text replacement. Every edits[].oldText must match a unique, non-overlapping region of the original file. If two changes affect the same block or nearby lines, merge them into one edit instead of emitting overlapping edits. Do not include large unchanged regions just to connect distant changes. *[click to show parameters]*

write - Write content to a file. Creates the file if it doesn't exist, overwrites if it does. Automatically creates parent directories. *[click to show parameters]*

05:32:32 PM

Switched to model: ollama/qwen3.6:35b-mlx

06:19:08 PM

sumamrize whats all in hii

06:19:22 PM

The user wants a summary of the directory named "hii". I should first check if this directory exists and what's inside it using bash commands.

1. List the contents of the "hii" directory to see its structure.
2. Based on the contents, read relevant files or list subdirectories if necessary.
3. Summarize the findings for the user.

First step: ``ls -la hii`` or similar command. Then I'll explore further based on what I find.

```
$ ls -la hii
```

```
ls: hii: No such file or directory
Command exited with code 1
```

06:19:24 PM

The directory "hii" does not exist in the current working directory. I should check if it exists elsewhere or if there's a typo by looking at the files in the current directory and maybe checking if ``hii`` is in the root or another common location, but since the user said "in hii", it implies they think it exists. Let me first check what `*is*` in the current directory to be helpful, and then perhaps mention that "hii" isn't there. It's possible it's a large filename or a typo for ``hi`` or something. Or maybe I need to look at the whole filesystem. But usually, LLMs operate in ``/root`` or project folders. Let me check the current directory contents.

```
$ ls -la
```

```
total 7560
drwxr-xr-x  86 ummi  staff    2752 Aug 21 17:43 .
drwxr-x--x+ 244 ummi  staff    7808 Aug 21 18:19 ..
-rw-r--r--@  1 ummi  staff   12292 Aug 18 05:45 .DS_Store
-rw-r--r--   1 ummi  staff    606 Aug 16 10:32 .env.example
... (82 more lines)
```

06:19:48 PM

The user asked to "summarize what's all in hii". The directory ``hii`` does not exist. However, I found a folder named ``.hii-app``. It is also possible they meant ``hi*`` as in a glob or something else, but ``.hii-app`` seems most likely related to "hii" if there is a project context, OR maybe ``archive``? Or perhaps ``home``?

Let me look at the list again.

- ``.hii-app`` (directory)
- ``hi`` ? No exact match for ``hii``.
- Is it possible the user meant the root directory or "what's all in here"? "in hii" is very specific. If I cannot find ``hii``, I should mention that and perhaps ask if they meant ``.hii-app``.

Wait, let me check if there is a file/folder named `hi` inside?

I see `a...`, `app`, `archive`, `article`, ...

No exact `hii`.

Let me double check the spelling. "sumamrize whats all in hii". Maybe "hii" refers to the project name or something? The folder `.hii-app` suggests the project might be called `hi` or `hii`.

I will assume the user meant the `.hii-app` directory, or perhaps they made a typo for "here".

However, since I am an AI, and usually the prompt is precise, maybe `hii` is a specific file/directory that got missed?

Let me run `find . -name "\*hii\*"` just to be safe.

Also, let's look at `.hii-app`. It has 2 entries (64 bytes). That's very small.

Maybe they meant `home`? `archive`?

Actually, looking at the prompt "sumamrize whats all in hii", it looks a lot like "summarize what's all in *here*". If I say "There is no file named 'hii', did you mean 'here'?" that sounds silly.

Let's run `find . -maxdepth 1 -name "\*hii\*"`.

If nothing, I will list the top-level structure of the current directory (which looks like a Rust + Typescript project structure). But strictly speaking, if they asked for "hii", and it doesn't exist, I should report that.

Let me check `find . -iname "\*hii\*"`.

And also just quickly peek at `.hii-app` since it seems to be the only match containing the string "hii".

It's a hidden folder. Usually hidden folders are config or system specific.

Let's explore `.hii-app` and report back about "hii" not being found if that's the only trace.

Wait, looking at `src-tauri`, `next.config.mjs`, `Cargo.toml`, there is likely a Tauri + Next.js app.

The user might be testing me or referring to a previous context I don't have access to (though I should check if there's a file named `hii`).

Let's do a broader search first. Then I will summarize the current root directory structure just in case "hii" was a typo for "here". It's more helpful.

Let's start by verifying "hii" and looking at `.hii-app`.

And maybe check if there is a `hi` folder? (The user might have typed double 'i' instead of something else).

Actually, I will just explain that "hii" doesn't exist exactly, but point out `.hii-app`, and then give a summary of the *current root directory* since "here" and "hii" can be typos for each other in some contexts (though less likely visually), or perhaps they meant `home`. Actually, let's just look at the structure. It looks like a dev workspace.

Plan:

1. Check if "hii" is a file/folder anywhere.
2. If not, summarize the current directory root as if asking "what's here". (This handles the typo hypothesis).

3. Mention `.hii-app` specifically in case that was the target.

Let's execute `find` first. Then a tree of root.

Wait, I can just summarize the root directory directly. The list output above is comprehensive enough for me to categorize it (Rust/Tauri, Next.js/React, Scripts, Docs).

I will clarify that "hii" wasn't found but provide a summary of the current folder structure. Actually, I'll check `.hii-app` content first, as it's a specific match.

Then I'll summarize the root directory (which starts with `.` and common dev folders).

Let's peek inside `.hii-app`. It has 64 bytes, likely just files or empty. `ls -la .hii-app` will tell me more than just line count.

I have enough info to form a hypothesis: The user likely meant "what's all in here".

I will provide a summary of the current directory listing.

Categories visible:

- ****Configuration/Boilerplate****: `.env.example`, `package.json`, `tsconfig.json`, `Cargo.toml`, `wrangler.jsonc`.
- ****Frameworks****: Rust (`src-tauri`), Node.js/Next.js (`app`, `next.config.mjs`).
- ****Code Structure****: `components`, `lib`, `scripts`, `server`, `workers`.
- ****Assets/Media****: `public`, `fabric`, `garden`, `artifacts`.
- ****Docs/Business****: `README.md`, `CLA.md`, `CONTRIBUTING.md`, `SECURITY.md`.

I will provide this summary. I should also mention that the specific folder "hii" was not found, but `.hii-app` exists.

Let me double check if there is a file named `hi`. No.

I will proceed with the "what's here" summary + note about missing "hii".

Wait, looking at the user request again: `sumamrize whats all in hii`. Maybe `hii` refers to a specific folder I missed? `home`? `app`?

What if it is `.hi`? No.

I'll assume typo for "here".

Let's just generate the response based on the file listing I already have. It's very detailed.

Structure:

1. Clarification: No file/folder strictly named "hii" found (mentioned `.hii-app`). Assuming you meant "here".
2. Summary of the project structure visible in `ls -la`.

The listing shows:

- ****Root Directories****: `app` (Next.js), `components`, `lib`, `scripts`, `server`, `skills`, `workers`.
- ****Frontend/UI****: `.next`, `public/`, `fabric` (likely assets).
- ****Backend/Rust****: `src-tauri` (Tauri App), `crates`.
- ****Config****: `package.json`, `Cargo.toml`, `tsconfig.json`.
- ****Documentation****: `README.md`, `CLA.md`, `CONTRIBUTING.md`.
- ****Data/Assets****: `garden`, `artifacts`, `notes`, `records`.

I will write a summary of the current directory.

(Self-correction): What if the user is referring to `home` and typed `hii` because of some keyboard mapping or autocorrect? Or `hi`?

Whatever, "here" is the most robust assumption for a summary request with a typo'd demonstrative.

I will structure the answer:

1. State that "hii" wasn't found (though `.hii-app` exists).
2. Summarize the current directory (The `ummi` root project) based on the listing provided.

Key observations from the list:

- **Framework**: Next.js (`app`, `components`) and Rust Tauri for native desktop apps (`src-tauri`). Also Wrangler (`wrangler.jsonc`) suggests Cloudflare Workers.
- **TypeScript/Node**: `node\_modules`, `package-lock.json`.
- **Documentation**: Several `.md` files (CLA, Contributing, Security).
- **Utilities**: A python script (`remove\_orange\_bg.py`), scripts directory.
- **Data**: `archive`, `contacts`, `expenses`.

I'll formulate the response now.

```
$ ls -la .hii-app
```

```
total 0
drwxr-xr-x  2 ummi  staff   64 Aug 17 03:32 .
drwxr-xr-x 86 ummi  staff 2752 Aug 21 17:43 ..
```

06:19:55 PM

The `.hii-app` folder is empty.

Now I am very confident the user meant "here" (the current working directory).

I will provide a structured summary of the files and directories currently in the workspace.

Categories to use for the summary:

- **Projects & Code**: Next.js app (`app`, `components`), Rust/Tauri backend (`src-tauri`, `crates`), Scripts (`scripts`).
- **Configurations**: `.env.example`, `package.json`, `Cargo.toml`, `tsconfig.json`.
- **Documentation**: README, CLA, Contributing guides.
- **Data/Assets**: Archive, Artifacts, Garden (likely images/media).
- **DevOps/Tech**: Node modules, Wrangler config (Cloudflare Workers).

I will present this in a clear, bulleted list.

It appears there is no file or folder specifically named `hii` in this location (though there is an empty `.hii-app` directory).

Assuming you meant "what's all in here?", here is a summary of the project structure located in this root directory:

Overview:

This is a complex development workspace, likely for a **Tauri-based desktop application** running on **Next.js** (React) with a Rust backend. It includes configurations for Cloudflare Workers and extensive documentation.

Key Directories & Components:

- **Application Frontend:**

- `app/`: Contains the Next.js UI components.
- `components/`, `lib/`: Shared frontend code and libraries.
- `public/`: Static assets.

- **Backend & Native Integration:**

- `src-tauri/`, `crates/`: The Rust-based native app logic using the Tauri framework.
- `server/`: Backend server code.
- `workers/`: Likely Serverless or Cloudflare Worker scripts.

- **Scripting & Tools:**

- `scripts/`: Various automation scripts.
- `remove_orange_bg.py`: A Python script for image processing (likely for assets).

- **Data & Assets:**

- `archives/`, `records/`: Storage for past data or logs.
- `fabric/`, `artifacts/`: Likely contains generated files, designs, or templates.
- `garden/`, `contacts/`: Folders suggesting personal or user-specific data structures.

- **Configuration & Docs:**

- `README.md`, `CLA.md`, `CONTRIBUTING.md`: Documentation and legal agreements.
- `package.json`, `Cargo.toml`: Configuration for Node.js (NPM) and Rust dependencies.

Is there something specific you were looking for, or perhaps another folder name?

06:21:34 PM

Switched to model: `ollama/qwen3.8:27b-mlx`

06:21:46 PM

summarize the current state of hii

06:23:09 PM

The user wants to summarize the current state of "hii" – and per the AGENTS.md guidance, HII is at `/Users/ummi/hii`. My current working directory is `/Users/ummi/hii`. Wait, the ls showed files like `src-tauri`, `cli`, `fabric`, `aii` etc. – this IS the hii repo. Let me use the canonical entrypoint `hii home --json` as AGENTS.md instructs, plus check git status. Let me run these probes.

Let me try `hii home --brief` first (cheap ~600 bytes) or just run the json. Per AGENTS.md, orient with `hii home --brief` first. Let me check if the hii binary exists at `target/debug/hii`.

You're right – this is the `hii` repo. Let me use the canonical entrypoints to get its live state rather than just reading the tree.

```
$ ./target/debug/hii home --brief 2>&1 | head -50; echo "---EXIT $?---"; ls -la
./target/debug/hii 2>&1
```

```
{"kind":"hii.agent.home.brief","repo":"/Users/ummi/hii","branch":"main","clean":false,"changes":72,"openTasks":7,"activeJobs":0,"capabilities":41,"nextActions":
[{"score":88,"track":"local CI/CD","action":"run build and CLI checks after each meaningful implementation step"},{"score":76,"track":"Context Dock truthful bootstrap","action":"align product truth, storage, permissions, context provenance, and bounded MCP around the Context Dock vertical slice"},{"score":62,"track":"Termite capability","action":"keep Termite as the first proof capability with logs and artifacts"}]]
---EXIT 0---
-rwxr-xr-x  1 ummi  staff  43756488 Aug 21 13:27 ./target/debug/hii
```

```
$ git status --short 2>&1 | head -40; echo "---"; git branch --show-current 2>&1; git log --oneline -5 2>&1
```

```
M app/globals.css
M app/layout.tsx
M app/page.tsx
M cli/src/main.rs
M components/workspace/HiiRoot.tsx
... (42 more lines)
```

06:23:17 PM

Now let me get the deeper home json to get the actual work queue and capabilities. Let me run `hii home --json` and a few targeted probes. Also the CURRENT_STATE.md doc would be valuable. Let me read that too.

Good – I have a live picture. Let me pull the deeper home projection and the current-state doc.

```
$ ./target/debug/hii home --json 2>&1 | head -100
```

```
{
  "schemaVersion": 1,
  "kind": "hii.agent.home",
  "generatedAt": "2026-08-21T22:23:17.870Z",
  "identity": {
... (95 more lines)
```

```
$ cat docs/CURRENT_STATE.md 2>&1 | head -80
```

```
# HII – Current State
**Audit date:** 2026-08-20
**Commit:** `54ac1049` (branch `main`)
**Method:** executable-code inspection plus commands actually run on this machine.
```

... (75 more lines)

06:23:42 PM

I now have a solid picture. Let me get the remaining work tasks and capabilities to round out the summary. Let me get the rest of the home json (the work board and capabilities and next actions).

Let me get the remaining work tasks and capability summary.

```
$ ./target/debug/hii home --json 2>&1 | python3 -c "
import json,sys
d=json.load(sys.stdin)
print('=== OPEN TASKS ===')
for t in d.get('work',{}).get('board',{}).get('recent',[]):
    print(f"[{t.get('lane','?'):>8}] ({t.get('priority','?')}) {t.get('title','')} owner=
{t.get('owner','?')} coord={t.get('coordinate','')}\n")
print()
print('=== NEXT ACTIONS ===')
for a in d.get('nextActions',[]):
    print(f"score {a['score']:>3} {a['track']}: {a['action']}\n")
print()
print('=== CAP SUMMARIES (if any) ===')
caps=d.get('capabilities') or d.get('work',{}).get('capabilities')
print(json.dumps(caps, indent=2)[:1500] if caps else 'n/a in brief')
"

=== OPEN TASKS ===
[ backlog] (high) Create NJIT Kepler assignment importer for portfolio owner=main agent
coord=/Users/ummi/dev/website/umminuriddingreen.com-clean
[ backlog] (normal) Improve agent fluency with digital music production tools owner=main
agent coord=/Users/ummi/AGENTS.md + /Users/ummi/.hii/music-tools
[ next] (high) Turn cafe vlog chaos into a 7-day content pack owner=HII Desk coord=/
[ next] (normal) --json stream llama3.2:1b with live panel owner=operator
coord=/Users/ummi/hii-newest
... (101 more lines)
```

```
$ ./target/debug/hii health --text 2>&1 | head -40; echo "=== CAPS COUNT ===";
./target/debug/hii caps show 2>&1 | tail -3
```

```
HII - Human Information Interface status
repo:    /Users/ummi/hii
git:     main
... (17 more lines)
```

06:23:57 PM

Now I have enough to write a good summary. Let me get the full caps count quickly.

```
$ ./target/debug/hii caps show 2>&1 | grep -cE "ready|partial|stale|missing" ; echo "---
status counts ---"; ./target/debug/hii caps show 2>&1 | grep -oE "
(ready|partial|stale|missing|unavailable)" | sort | uniq -c
```

```
42
--- status counts ---
   8 partial
  34 ready
   1 unavailable
```

06:25:07 PM

Now I have a comprehensive picture. Let me write the summary.

Key facts:

- Repo: /Users/ummi/hii, branch main, NOT clean, 72 changes (16 modified, 56 untracked), 0 staged/committed of the new work
- 7 open tasks, 0 active jobs
- 41 capabilities (one unavailable? Actually 42 shown by grep but caps show shows "capabilities: 41" in brief – let me reconcile: brief said 41, caps show grep found 42 status lines. Maybe one is a header or the count includes something. I'll say ~41-42, 34 ready / 8 partial / 1 unavailable).
- CURRENT_STATE.md says Wave 1-5 of Space work implemented, integrated gate passing (88 vitest / 560 tests, 386 Rust tests), but npm audit --omit=dev is RED (high-severity advisories, breaking Next upgrade not applied).
- The untracked files show new work: ecosystem/ and spaces/ subsystems, docs, fabric/identity, fabric/relay, system-self-audit.mjs
- env: several MISSING secrets (Supabase, R2, Stripe, BASE_URL) – but those may be expected on this operator machine (config reference only)
- health: codex-cli 0.148.0, bridge present

Let me write a concise but informative summary with the coordinate-rich shape per AGENTS.md guidance.

HII current state (verified live, 2026-08-21 22:23)

Coordinate: /Users/ummi/hii · branch main · binary ./target/debug/hii (built, working) · runtime /Users/ummi/.hii

What it is

A local-first "human information interface" – a CLI-first control plane (Rust) with Tauri web/Tauri/spatial projections over CLI-owned state. Product wedge: *object + intent* → *bounded capability* → *result* → *proof* → *durable receipt*.

Working tree – DIRTY

- **72 uncommitted changes:** 16 modified, 0 staged, 56 untracked, nothing pushed past commit 54ac1049.
- The untracked set is a whole new feature area, not cleanup:
 - **Spaces subsystem** – lib/spaces/, lib/fabric/, components/spaces/, lib/server/space-store.ts, scripts/hii-spaces-*.mjs, docs/SPACES_*.md (5 docs).

- **Ecosystem subsystem** – `cli/src/ecosystem_catalog.rs`, `components/ecosystem/`, `lib/ecosystem/contracts.ts`, `docs/ECOSYSTEM_*.md`.
- `fabric/identity/`, `fabric/relay/`, `aii/scripts/system-self-audit.mjs`.

- **This work is uncommitted** – all the new subsystems live only in the worktree.

Work queue – 7 open, 0 doing, 0 blocked

lane	task	owner
next/h	Design authoritative auto-model router (config-backed, no auto-escalation)	operator
next/n	Rotate Apple Sign-In web secret before 2027-01-18	operator
next/n	Cafe vlog → 7-day content pack	HII Desk
next/n	<code>--json</code> stream llama3.2:1b live panel	operator
next/n	receipt smoke test	screenctx
backlog	NJIT Kepler importer (portfolio)	main agent
backlog	digital music agent tools	main agent

Capabilities – ~41 registered

34 ready • 8 partial • 1 unavailable. Notable partials: `hii.agent.spawn`, `hii.space.desktop_control`, `termite.rhino.managed_job`.

Verification state (from `docs/CURRENT_STATE.md`, dated 2026-08-20 – now slightly stale vs. worktree)

- **Green:** 560 Vitest / 386 Rust tests, typecheck, lint, static build, Space+Workspace persistence smokes, secret scan.
- **RED:** `npm audit --omit=dev` – high-severity advisories in Next/PostCSS/Nanoid; fix is a **breaking major upgrade, not yet applied**.
- **Open risk:** Spaces LAN proof ran on this Mac's `en0`, but a **physical second-device / phone smoke is still unverified**.

Environment

Missing secrets: Supabase (3), R2 (4), Stripe (2), `NEXT_PUBLIC_BASE_URL` – likely config-reference-only on this box, but flag if any surface needs them. `codex-cli 0.148.0`, bridge file present.

Top next actions (by HII's own scoring)

1. (88) local CI/CD – run `cargo build/test` + CLI checks after each step.
2. (76) Context Dock truthful bootstrap.
3. (62) keep Termite as first proof capability.

Headline: the product is structurally healthy and its gates pass, but you have **two new subsystems** (Spaces, Ecosystem) sitting entirely uncommitted on `main`, a red `npm audit`

awaiting a breaking upgrade, a phone-reachability proof owed, and ~13 tasks queued with none in flight. Nothing is currently being worked (0 active jobs).

Want me to commit/stage the Spaces+ecosystem work, fix the audit advisories, or verify the LAN/phone smoke?